Since 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Engineering and Technology

MINI PROJECT REPORT

Computer Networks Laboratory

(CSE20110)

Laboratory Continuous Assessment 3

MPFT: Multipath Reliable File Transfer over Parallel TCP Paths

Submitted by

Sr. No.	Student Name	PRN
1	Saksham Singh Sisodiya	1262242908
2	Parth Dande	1262242629

Group No. [GROUP NUMBER] Panel [PANEL] Batch [BATCH]

Third Year B.Tech. Computer Engineering and Technology

Under the guidance of

Dr. Priti Chakurkar

Academic Year 2026–2027

Submission Date [DD MONTH YYYY]

Department of Computer Engineering and Technology

CERTIFICATE

This is to certify that the mini-project entitled “MPFT: Multipath Reliable File Transfer over Parallel TCP Paths” has been carried out by the following students of Third Year B.Tech. Computer Science and Engineering, Panel [PANEL], Batch [BATCH], Group No. [GROUP NUMBER], under my guidance during the academic year 2026–2027.

Student Name	PRN
Saksham Singh Sisodiya	1262242908
Parth Dande	1262242629

The report is submitted towards the mini-project component of Laboratory Continuous Assessment 3 for the Computer Networks Laboratory course.

Faculty Signature _____

Dr. Priti Chakurkar

Project Guide and Course Faculty

Date _____ Place: Pune

Acknowledgement

We thank our course faculty, Dr. Priti Chakurkar, for guidance on the Computer Networks Laboratory mini-project and for the assessment criteria that shaped the scope of this work. We also thank the Department of Computer Engineering and Technology for the laboratory facilities. The technical resources we relied on are the Python standard library documentation, the TCP and Multipath TCP RFCs, and the Wireshark documentation listed in the References. We used an AI coding assistant during development; this is described in the AI Usage Declaration.

Group No. [GROUP NUMBER]

Abstract

Transferring a large file over a single TCP connection leaves other available network paths unused and restarts from zero when the connection fails. This project designs and implements MPFT, a Multipath Reliable File Transfer protocol that splits a file into chunks and sends them in parallel over several TCP connections, one per path. Paths take chunks from a shared queue, so faster paths automatically carry more data. Each chunk is checked with SHA-256, the whole file is verified before it is renamed, and a failed, silent or corrupting path only causes its chunks to be retransmitted over the others. An interrupted transfer resumes by skipping verified chunks. The system uses only the Python standard library; paths are emulated by a TCP proxy that adds delay, jitter, rate limits, failures and corruption. A browser dashboard and a Wireshark dissector expose the protocol. On a single machine with emulated 40 Mbit/s paths, goodput grew from 23.0 to 171.9 Mbit/s from one to eight paths, every transfer, including one with a path killed mid-transfer and runs with 5 % corrupted chunks, finished with a matching SHA-256, and a resumed run skipped the 80 chunks already held.

Keywords: Multipath transfer, TCP, reliable delivery, work stealing, SHA-256, network emulation, Wireshark

Table of Contents

Acknowledgement **3**

Abstract **3**

List of Figures **5**

List of Tables **5**

1 Introduction and Project Objectives **6**

1.1 Introduction and Background 6

1.2 Problem Statement 6

1.3 Scope 6

1.4 Objectives 7

2 Proposed Design and Requirements **8**

2.1 Architecture and Network Topology 8

2.2 Tools and System Requirements 8

2.3 Addressing and Protocol Selection 9

3 Implementation and Technical Working **11**

3.1 Methodology 11

3.2 Configuration or Code Implementation 11

3.3 Protocols and Technical Working 12

3.4 Execution Procedure 12

3.5 Sender and Receiver Views, Scenarios and Graph Axis 15

4 Testing and Results **19**

4.1 Test Plan 19

4.2 Results and Execution Evidence 20

4.3 Objective Achievement 23

5 Discussion and Conclusion **25**

5.1 Discussion and Comparative Analysis 25

5.2 Advantages and Applications 25

5.3 Limitations 25

5.4 Challenges and Troubleshooting 26

5.5 Future Scope 26

5.6 Conclusion 27

References **28**

AI Usage Declaration **29**

Appendices **30**

Appendix A Code and Configuration 30

Appendix B Additional Evidence 31

Appendix C Individual Contributions 32

List of Figures

Figure 2.1 Proposed architecture: sender, parallel emulated paths, receiver 8

Figure 3.1 MPFT message sequence with a NACK and retransmission 11

Figure 3.2 Dashboard for a 30 MiB transfer with a path killed at 3 s 14

Figure 3.3 Per-path table, incidents and file verification 15

Figure 3.4 Sender view: Delivered, with the scenario and transfer figures 15

Figure 3.5 Receiver view: Saved and verified, with Download 16

Figure 3.6 Receiver view of a failed transfer (total failure scenario) 17

Figure 3.7 Slow-WAN scenario: per-path graphs on a millisecond axis 18

Figure 4.1 Goodput versus number of paths (each 40 Mbit/s, 10 ms) 21

Figure 4.2 Transfer time versus extra delay on one of two paths 21

Figure 4.3 Bytes carried per path (80 / 20 / 5 Mbit/s) 22

Figure 4.4 Recovery timeline after a path failure 22

Figure 4.5 Retransmitted chunks versus corruption rate 23

Figure 4.6 Goodput versus chunk size 23

List of Tables

Table 1.1 Project objectives and verification 7

Table 2.1 Tools and resources 9

Table 2.2 Addresses and ports (topology.example.json defaults) 9

Table 2.3 MPFT header (20 bytes, big-endian) 10

Table 4.1 Automated tests (pytest) 19

Table 4.2 Experiments on the emulated network (results/*.csv) 20

Table 4.3 Objective achievement 24

Table 5.1 Problems found and how they were resolved 26

Table A.1 Main source files 30

Table B.1 Selected raw results 32

Table C.1 Individual contributions 32

1 Introduction and Project Objectives

1.1 Introduction and Background

A single TCP connection delivers a reliable, ordered byte stream between two hosts [1]. It is reliable, but one connection follows one network path, so a host with Wi-Fi and Ethernet, or a path to a server with several routes, uses only one of them. Congestion control inside TCP also limits how quickly one connection can fill a path [2]. If the connection breaks half-way through a large file, a simple client has to start again.

Multipath TCP (MPTCP) addresses this at the kernel level by spreading one connection across several subflows [3]. The deployment study by Raiciu et al. shows how hard it is to make such a protocol work through real middleboxes [4]. MPTCP needs support in both operating systems and in the network, which makes it unsuitable as a classroom experiment. A user-space alternative is to open several ordinary TCP connections and divide the file between them. This is what MPFT does, and the division is the interesting part: static splitting wastes the faster paths, so the sender uses a shared work queue.

File integrity is a separate concern from delivery. TCP's checksum is weak against corruption that occurs outside the network, so MPFT also applies SHA-256 [5] to each chunk and to the whole file. The sections below describe the protocol, the sender and receiver, the network emulator used for testing, and measurements that were run on this implementation.

1.2 Problem Statement

There is no simple way for a student on one machine to see how a file transfer behaves when several paths have different speeds, delays and failure rates. Single-connection tools (scp, a plain socket program) cannot use more than one path, cannot continue after a lost connection and give little visibility into what the network did. The problem is to design and build a file-transfer protocol that (a) uses several paths at once, (b) delivers a bit-exact file despite path failure, silent paths and corruption, (c) resumes after interruption and (d) lets the user watch and capture the protocol.

1.3 Scope

MPFT transfers one file at a time between a sender and a receiver. Each path is one TCP connection; the paths are emulated on a single macOS machine by a Python TCP proxy

(mpft/emulator.py) that can add delay and jitter, limit the rate, kill the connection, or flip bits. Assumptions: the receiver is reachable on every address the sender uses; the file fits on the receiver's disk; and path conditions are those the emulator produces. Out of scope: encryption and authentication, IP-level packet loss (the proxy works above TCP), congestion control across paths, and multiple simultaneous senders. A browser interface and a two-machine mode over Tailscale or LAN were implemented, but the results in this report come from the single-machine emulated setup.

1.4 Objectives

Each objective is stated so that a test or measurement can confirm it.

Table 1.1 Project objectives and verification

Objective ID	Objective	Verification Method
O1	Transfer one file over N parallel TCP paths using a shared chunk queue.	Tests U6, E1: output SHA-256 equals input; goodput versus path count
O2	Detect corruption at header, chunk and file level and repair it by retransmission.	Tests U2, U7, U11, E5: CRC error, NACK, retransmission count, final hash
O3	Complete the transfer when a path dies or goes silent; retry at most 5 times per chunk.	Tests U9, U10, U12, E4: success flag, recovery time, retry limit
O4	Give faster paths more data without measuring bandwidth.	Tests U15, E2, E3: bytes per path
O5	Resume an interrupted transfer and skip chunks already verified on disk.	Tests U13, U14, E7: resumed chunk count, time
O6	Make the protocol observable (status files, dashboard, Wireshark dissector).	Tests U19, U20, web and e2e tests; tshark decoding of a loopback capture

2 Proposed Design and Requirements

2.1 Architecture and Network Topology

Figure 2.1 shows the components. The sender hashes the file in one streaming pass to build a manifest (size, chunk size, per-chunk SHA-256, whole-file SHA-256). It opens one TCP connection per path. Every path thread repeatedly takes the next chunk index from a shared priority queue, reads that chunk from disk with pread, sends it, and waits for an ACK or NACK. The receiver verifies each chunk, writes it at its offset in file.part with pwrite, and after the last chunk verifies the whole file and renames it. Neither side keeps the whole file in memory. Each side also writes a status JSON file every second, which the dashboard reads.

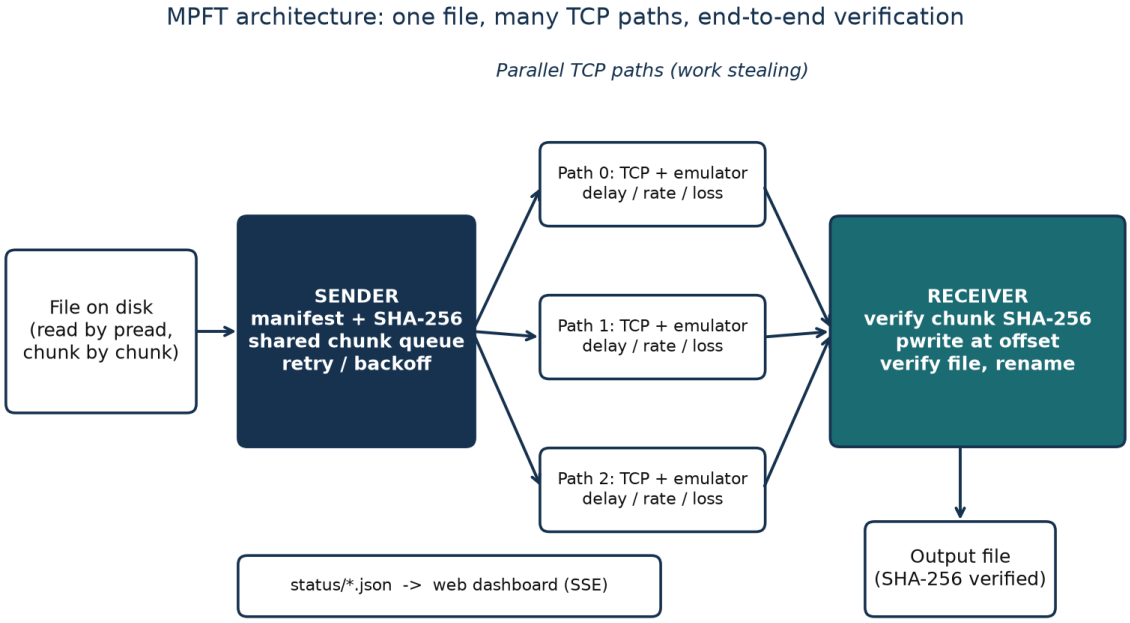


Figure 2.1 Proposed architecture: sender, parallel emulated paths, receiver

2.2 Tools and System Requirements

All measurements were taken on one MacBook (macOS 27.0.1). The protocol core uses only the Python standard library [6]; the extras are used for tests, plots and the web layer. The Wireshark dissector is written in Lua [7].

Table 2.1 Tools and resources

Tool or Resource	Version or Specification	Purpose
Python	3.15.0 (virtual environment)	Sender, receiver, emulator, CLI (standard library only)
macOS	27.0.1	Host operating system; loopback network
pytest	9.1.1	50 unit/integration tests (plus Playwright end-to-end tests)
matplotlib / numpy	3.11.2 / 2.5.4	Graphs from the experiment CSV files
FastAPI / uvicorn / httpx	0.148.0 / 0.54.0 / 0.28.1	Dashboard and web UI server, test client
Playwright	1.64.0	Browser end-to-end tests and UI screenshots
Wireshark / tshark + Lua	mpft.lua dissector	Protocol decoding of captured traffic
Headless Chrome	n/a	Markdown-to-PDF documentation

2.3 Addressing and Protocol Selection

MPFT is a socket project. Hosts are on loopback; the emulator listens on one port per path and forwards to the receiver. Table 2.2 lists the default addresses; there are no gateways because everything runs on 127.0.0.1 (the receiver can instead listen on 0.0.0.0 for a LAN or Tailscale run).

Table 2.2 Addresses and ports (topology.example.json defaults)

Device / Interface	IP / Prefix	Gateway	Role
Receiver	0.0.0.0:9000 (TCP)	n/a (loopback)	Accepts all path connections; writes the file
Emulated path 0	127.0.0.1:9201 to 9000	n/a	Proxy, 2 ms delay, no rate limit
Emulated path 1	127.0.0.1:9202 to 9000	n/a	Proxy, 20 ms delay, 20 Mbit/s
Emulated path 2	127.0.0.1:9203 to 9000	n/a	Proxy, 40 ms delay, 8 Mbit/s, dies at 6 s
Web UI	127.0.0.1:8000	n/a	Browser interface (local only)
Peer API	0.0.0.0:8001	n/a	/health, /addresses, /offer, /events/{session}
Rendezvous (optional)	0.0.0.0:9100	n/a	Short code to receiver ID lookup

- **Why TCP per path.** TCP gives each path ordered, retransmitted delivery, so MPFT does not re-implement loss recovery inside a path. Reliability at the file level is added on top: a chunk that is corrupted, or whose connection dies, is requeued.

Table 2.3 MPFT header (20 bytes, big-endian)

Field	Bytes	Meaning
magic	2	ASCII "MP"
version	1	1
type	1	1 HELLO, 2 MANIFEST, 3 MANIFEST_ACK, 4 CHUNK, 5 ACK, 6 NACK, 7 DONE, 8 ERROR
session	4	Transfer identifier
chunk_index	4	Chunk number (0 for control messages)
payload_len	4	Bytes following the header (limit 64 MiB)
crc32	4	CRC-32 of the first 16 header bytes

- **Why chunks and a shared queue.** Independent chunks can arrive on any path in any order; the receiver writes them at their offsets. A fast path returns to the queue sooner, so it takes more chunks without any bandwidth estimate (work stealing).
- **Stop-and-wait per path.** Each path has one chunk in flight. This keeps the protocol easy to explain but costs one round trip per chunk, which explains why one 40 Mbit/s path reached only 23 Mbit/s (Section 5.1).
- **Header.** A fixed 20-byte header carries a CRC-32 of its own first 16 bytes so that a damaged header is rejected immediately, before the length field is trusted.
- **Retry policy.** A chunk is attempted at most 5 more times with exponential backoff (50 ms doubling, capped at 1 s); retried chunks go ahead of fresh ones.

3 Implementation and Technical Working

3.1 Methodology

Work was done in phases, each ending with tests: (1) protocol, sender and receiver; (2) failure handling and resume; (3) network emulator; (4) experiments and graphs; (5) Wireshark dissector; (6) status files and dashboard; (7) a browser-based send/receive interface with a live two-sided dashboard. Figure 3.1 shows the message exchange for one transfer, including a corrupted chunk that is rejected and sent again.

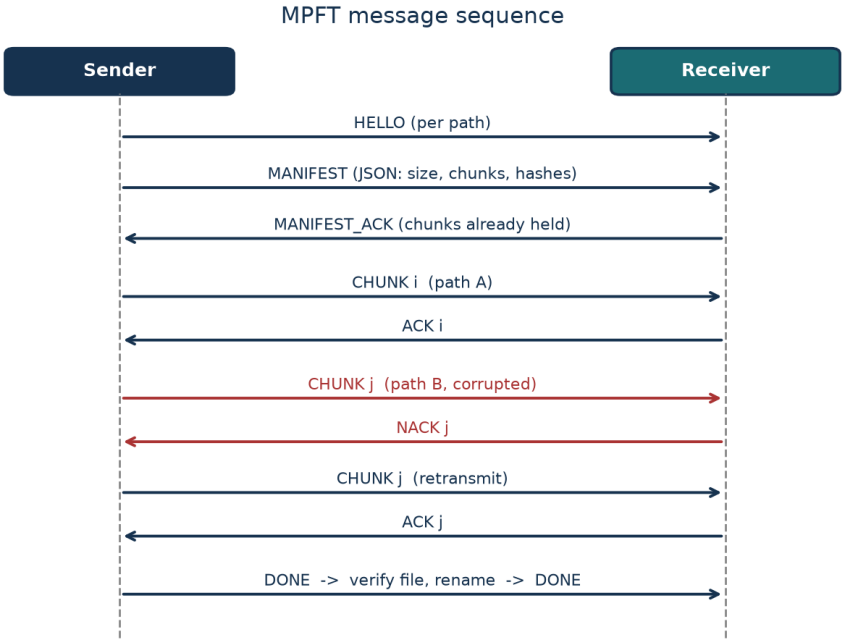


Figure 3.1 MPFT message sequence with a NACK and retransmission

3.2 Configuration or Code Implementation

The core is about 970 lines in nine files under mpft/, each small enough to read in one sitting. protocol.py defines the header and framing; sender.py the path threads, queue and retries; receiver.py verification, offset writes and resume; emulator.py the proxy; status.py the JSON status writer; cli.py the commands. The header functions are the basis of everything else:

```
def pack_header(mtype, session, index, payload_len):
    body = _BODY.pack(MAGIC, VERSION, mtype, session, index, payload_len)
    return body + struct.pack(">I", zlib.crc32(body))

def unpack_header(raw):
    body, (crc,) = raw[:-4], struct.unpack(">I", raw[-4:])
    if zlib.crc32(body) != crc:
        raise ProtocolError("header CRC mismatch")
```

The scheduler is described below in pseudocode (sender.py implements it with one thread per path).

```
queue = PriorityQueue()                  # items: (priority, chunk_index)
for i in chunks the receiver does not hold: queue.put((1, i))
path_worker(p):
    while not finished:
        (_, i) = queue.get(timeout=0.1)
        send CHUNK i; wait for reply (timeout 5 s)
        ACK     -> record bytes and RTT
        NACK    -> requeue(i)
        timeout / reset -> requeue(i); close socket; reconnect with backoff
requeue(i): attempts[i] += 1; fail if > 5; after min(0.05*2^(n-1), 1.0) s put (0, i)
```

3.3 Protocols and Technical Working

Connection setup. The sender opens one connection per path and sends HELLO; the first path sends the MANIFEST. The receiver answers MANIFEST_ACK listing the chunks it already holds and the sender queues only the rest.

Chunk delivery. CHUNK carries the chunk index in the header and the bytes as payload. The receiver recomputes SHA-256, compares it with the manifest, writes with pwrite and replies ACK, or replies NACK and stores nothing.

Failure handling. A NACK, a timeout or a closed socket puts the chunk back on the queue; the path reconnects with backoff and is marked dead after repeated failures.

Completion. When all chunks are acknowledged, the sender sends DONE. The receiver checks the whole-file SHA-256, renames file.part to the final name and answers DONE (or ERROR).

Resume. On start the receiver hashes any existing file.part chunk by chunk against the manifest, so damaged chunks are not trusted.

Emulator. For each path, a proxy thread pair relays bytes, sleeping for the delay (plus random jitter), pacing to the rate limit, closing after drop_after_s, or flipping bits per 16 KiB block. A small HTTP control port allows killing a path or changing corruption while running.

3.4 Execution Procedure

Prerequisites: macOS or Linux, Python 3.11 or newer. The commands below are the ones used in this project (from the repository root):

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
# terminal 1: receiver
.venv/bin/python -m mpft receive --listen 0.0.0.0:9000 --out downloads
```

```
# terminal 2: three emulated paths (see topology.example.json)
.venv/bin/python -m mpft emulate --config topology.example.json
# terminal 3: send a file over the three paths
.venv/bin/python -m mpft send big.bin --to
127.0.0.1:9201,127.0.0.1:9202,127.0.0.1:9203 \
    --chunk-size 256k
# tests and experiments
.venv/bin/pytest -q
.venv/bin/python experiments/run_experiments.py && .venv/bin/python experiments/
plot_results.py
# browser interface (run once per device)
.venv/bin/python -m mpft web --name alice --ui-port 8000 --peer-port 8001 --data-port
9000 --out ./downloads
```

Figure 3.2 shows the browser dashboard after a 30 MiB transfer in which one of two paths was killed 3 s in. The throughput graph shows path 1 dropping to zero, the incident marker records that path 1 went down and the transfer recovered, and path 0 delivered the remaining chunks.

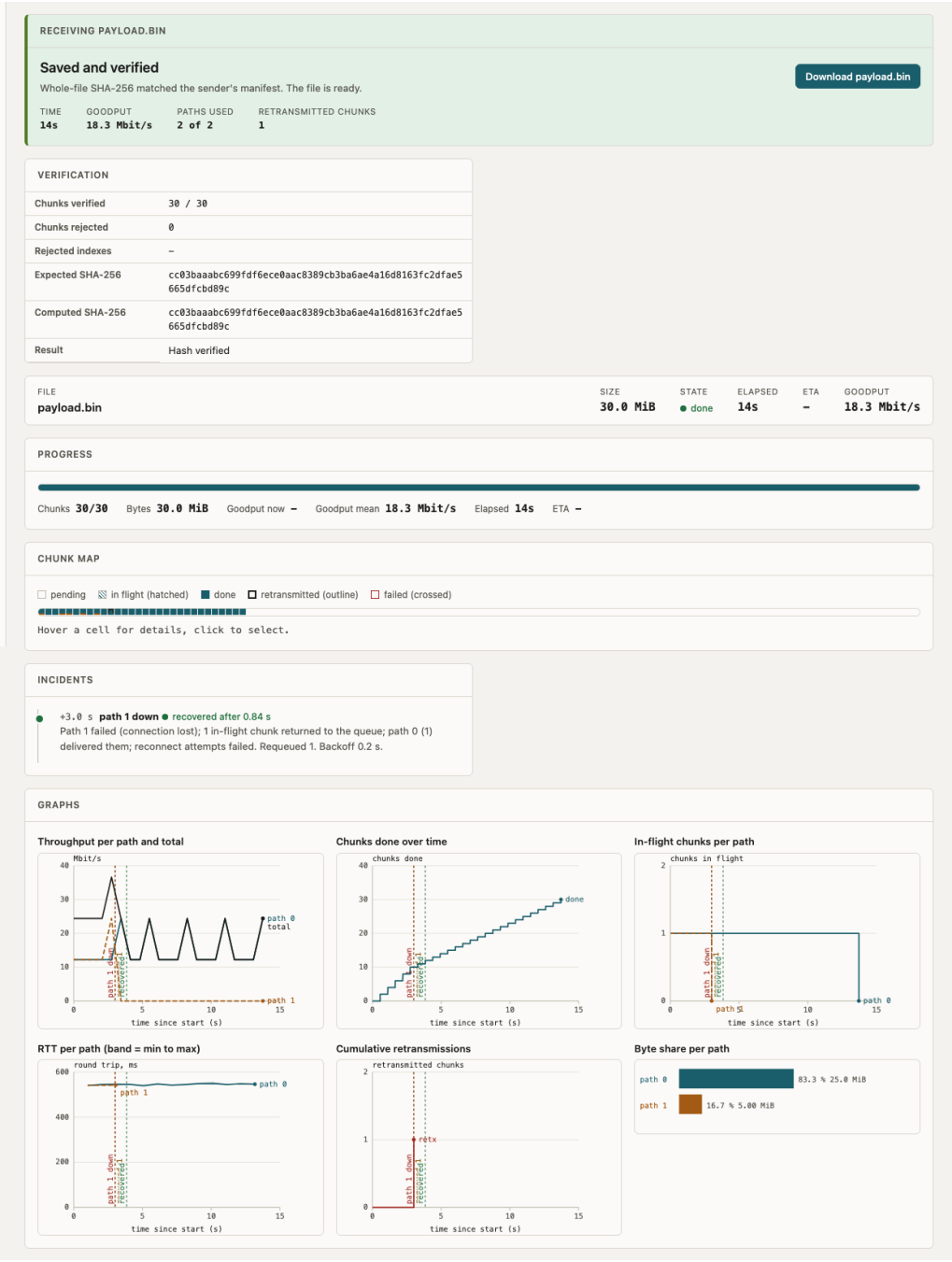


Figure 3.2 Dashboard for a 30 MiB transfer with a path killed at 3 s

Figure 3.3 is the lower part of the same page: per-path bytes and round-trip times, the incident description, and the final verification, where the expected and computed SHA-256 match.

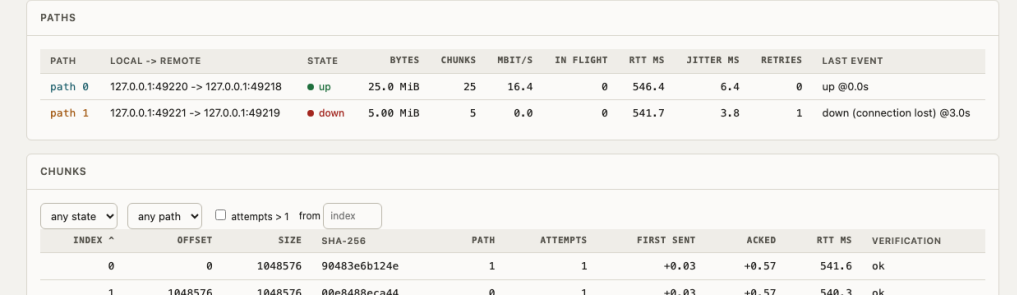


Figure 3.3 Per-path table, incidents and file verification

3.5 Sender and Receiver Views, Scenarios and Graph Axis

Both devices open the same session, but the page is role-aware. The title says Sending or Receiving, a coloured banner states the outcome for that side, and the panels are ordered for what each side needs. Figure 3.4 is the sender after a successful transfer: it reports delivery and offers to send another file. Figure 3.5 is the receiver for the same session: verification comes first and the Download button appears only here.

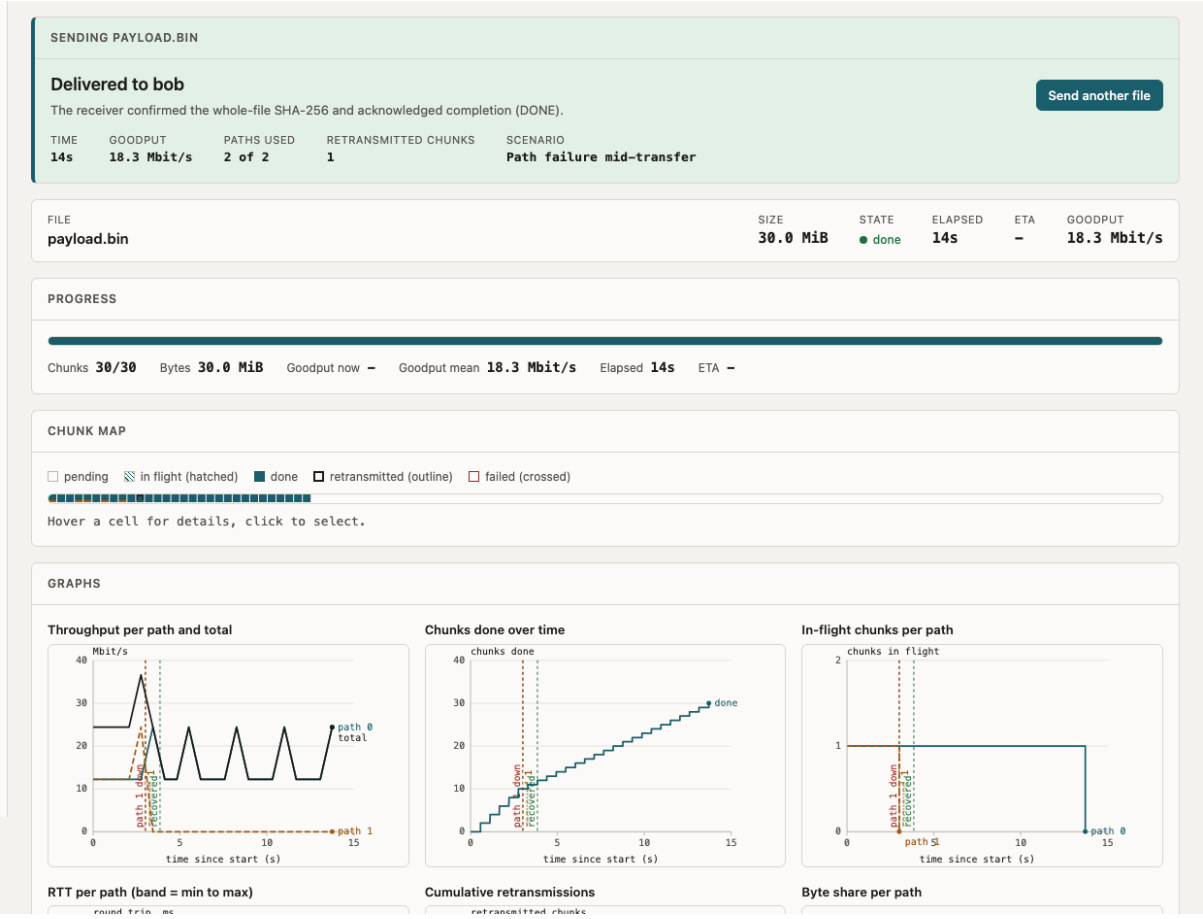


Figure 3.4 Sender view: Delivered, with the scenario and transfer figures

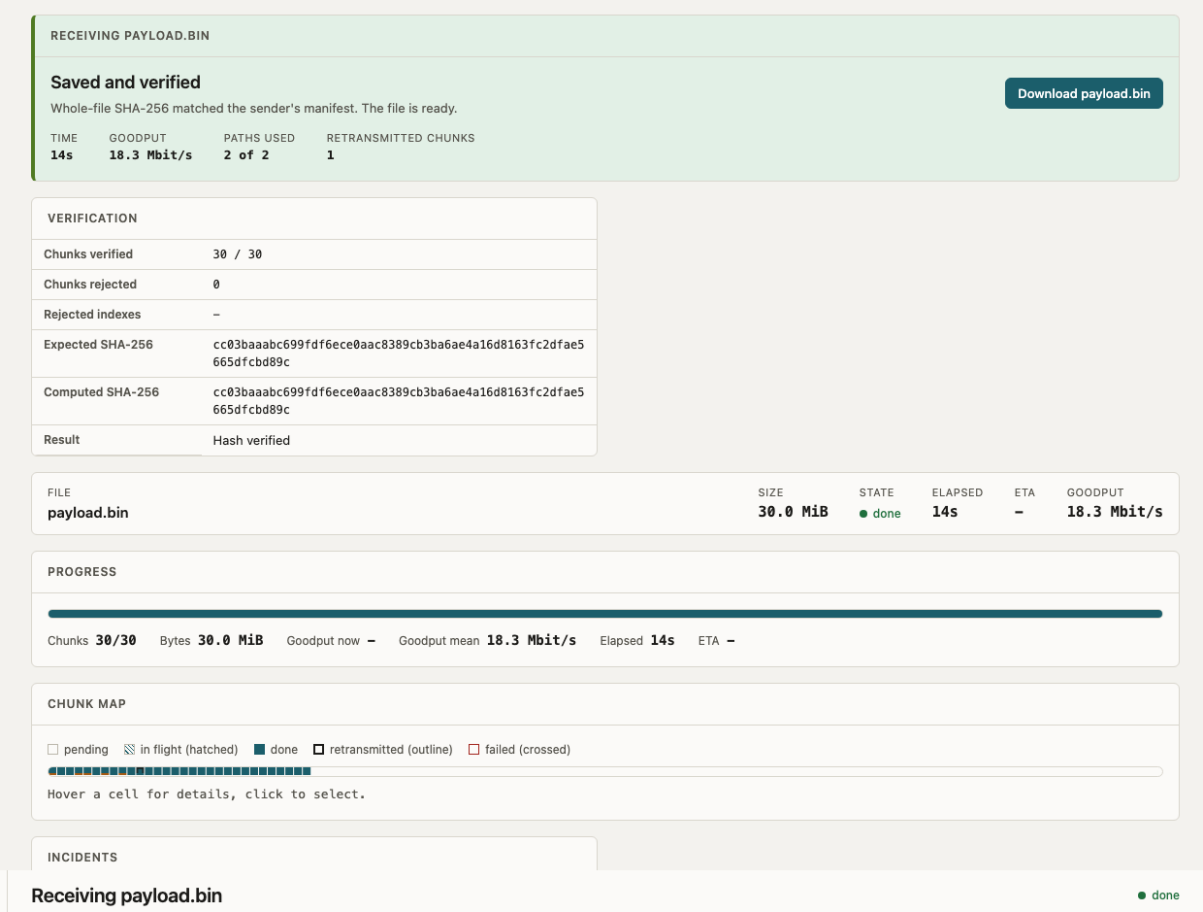


Figure 3.5 Receiver view: Saved and verified, with Download

Transfer scenarios. The Send form offers eight network scenarios, each run through the emulator in front of the receiver. This makes it easy to show success and failure side by side in a demonstration:

- Clean network (no emulation) and Clean but slow (16 Mbit/s per path, 20 ms delay): smooth, long curves.
- Asymmetric paths: faster paths carry more chunks (work stealing).
- Lossy (5% of chunks corrupted) and Heavy loss (20%): NACKs, retransmissions, and a matching final SHA-256.
- Path failure (last path dies after 3 s) and Path flaps (one path dies at 2 s and returns 3 s later): requeue, backoff and reconnect.
- Total failure (every path dies after 1 s, file over 2 MB): the transfer fails, and nothing is saved or offered for download.

Figure 3.6 shows the failure case on the receiving side. The banner reads Nothing was saved, the incident list shows each path going down, and no Download button is offered because no verified file exists.

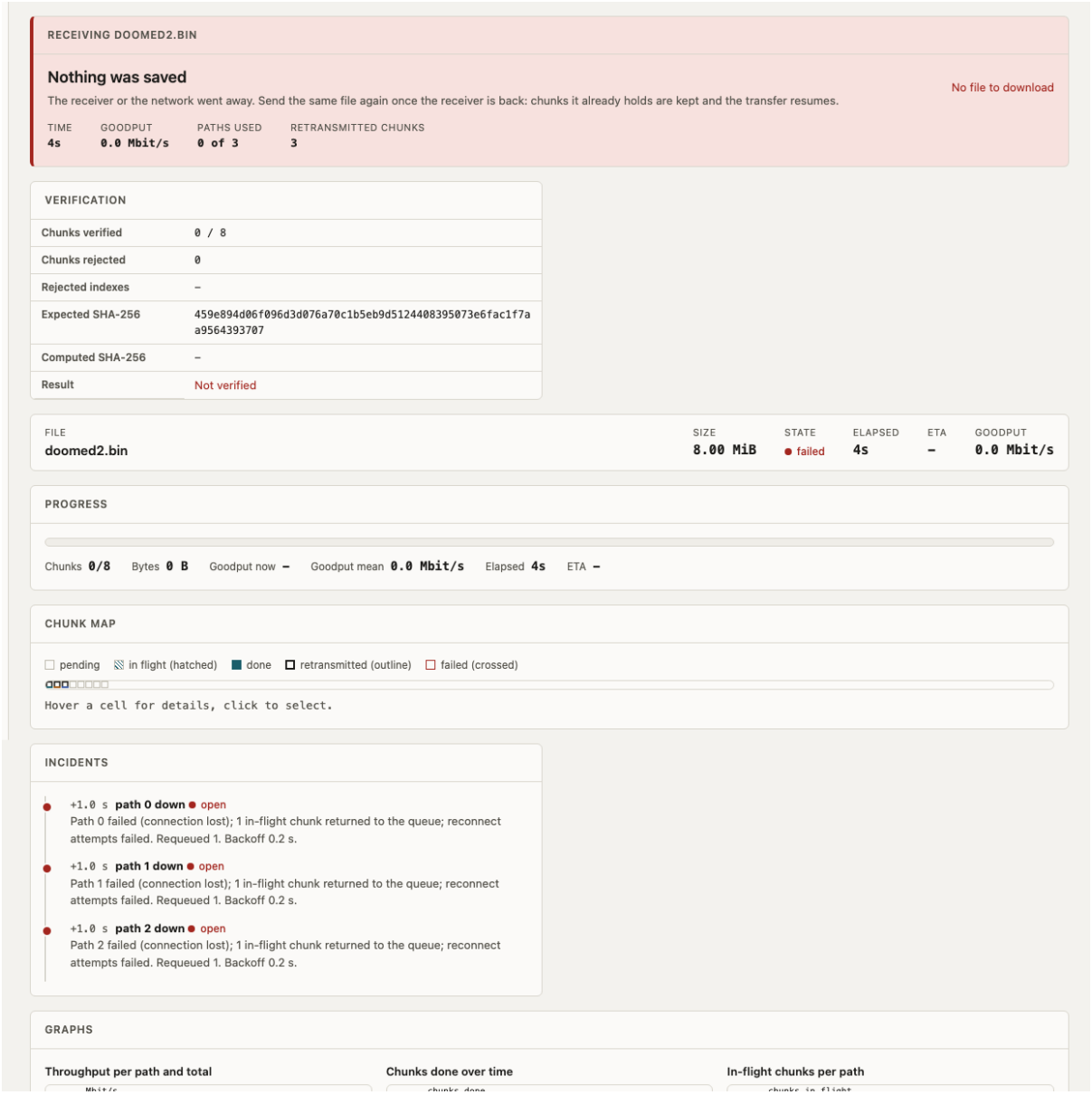


Figure 3.6 Receiver view of a failed transfer (total failure scenario)

Graph time axis. Transfers on loopback can finish in well under a second, which left the earlier graphs almost empty on a fixed seconds axis. Throughput is now computed in the browser from chunk completions in buckets of one twentieth of the transfer duration (one second only for transfers over 20 s), the x range equals the real duration, and transfers shorter than 2 s use a millisecond axis. Figure 3.7 shows the slow-WAN scenario, where per-path throughput, chunk completion and in-flight chunks are readable.

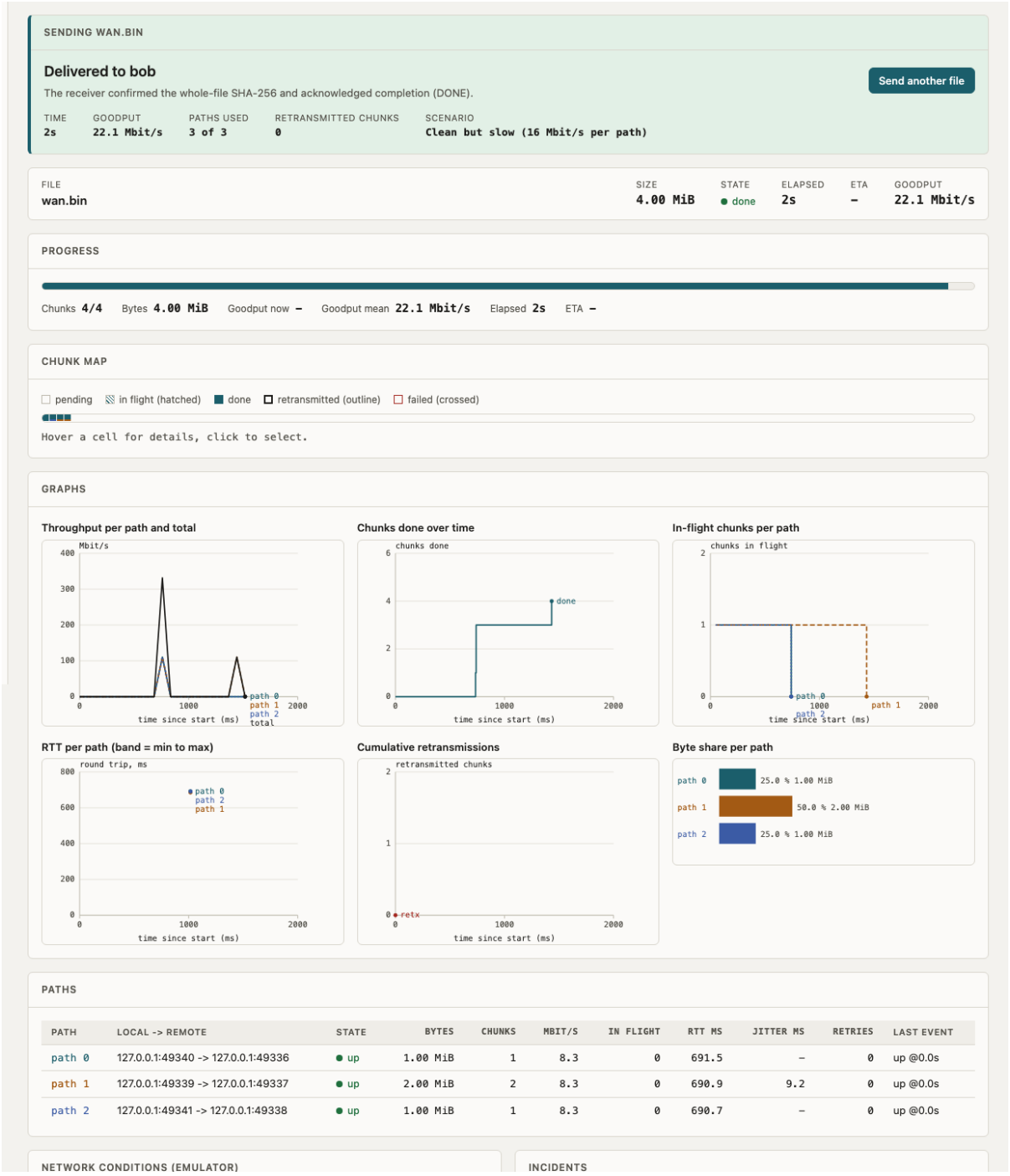


Figure 3.7 Slow-WAN scenario: per-path graphs on a millisecond axis

4 Testing and Results

4.1 Test Plan

Environment: one Mac, loopback, Python 3.15, paths emulated by mpft/emulator.py (default 40 Mbit/s and 10 ms one-way delay). Unit and integration tests run with pytest; experiments use a 50 MB random file and 256 KiB chunks unless stated, and each experiment was run once. Success means the output file's SHA-256 equals the input's, no file.part is left, and the sender reports success. The last full run of the non-browser test suite gave 50 passed, 1 skipped. Table 4.1 lists the main automated tests; the experiments are in Table 4.2.

Table 4.1 Automated tests (pytest)

Test ID	Test and Input	Expected Result	Observed Result	Status
T1	U1–U3: header round trip, flipped byte, wrong magic	20 bytes; CRC / magic errors raised	As expected; ProtocolError	Pass
T2	U4–U5: message with payload; frame delivered 1 byte at a time	Same fields; reassembled	As expected	Pass
T3	U6: 20 MB over 4 paths	SHA-256 equal; no .part left	Match; all bytes counted	Pass
T4	U7: chunk with one flipped bit	NACK; nothing stored; resend ACKed	As expected	Pass
T5	U8: duplicate and out-of-order chunks	All ACKed; file correct	As expected	Pass
T6	U9: path killed after 1 MiB	Other path finishes; failure recorded	Success, hash match	Pass
T7	U10: path swallows data (0.4 s timeout)	Timeout; other path finishes	Success, hash match	Pass
T8	U11: corrupt_rate 0.1 on 2 paths	Retransmissions > 0; file correct	As expected	Pass
T9	U12: corrupt_rate 1.0	Give up after 5 retries; success False	Error mentions retries	Pass
T10	U13–U14: resume; damaged stored chunk	Only missing chunks sent; damaged not trusted	Half resumed; 15 of 16 with damage; hash match	Pass
T11	U15: fast (1 ms) and slow (20 ms, 8 Mbit/s) path	Fast path carries more than 2x	As expected	Pass
T12	U16–U18: rate limiter, drop_after_s, emulator corruption	About 1 s for 1 MiB at 8 Mbit/s; survivor finishes	As expected	Pass
T13	U19–U20: dashboard with real and junk status files	Sessions listed; junk skipped; old marked stale	As expected	Pass
T14	Web flow: 10 MB via ID and via code; wrong, expired, used code; decline; size limit	Transfer completes; bad codes rejected	As expected	Pass

Table 4.2 Experiments on the emulated network (results/*.csv)

ID	Scenario	Expected	Observed	Status
E1	1, 2, 4, 8 equal paths	Goodput grows with paths	23.0 / 45.5 / 89.2 / 171.9 Mbit/s (18.2 s to 2.4 s)	Pass
E2	Paths 80 / 20 / 5 Mbit/s	Faster paths carry more	74 % / 20 % / 6 % of bytes; 7.6 s	Pass
E3	2 paths; one with 0 / 25 / 50 / 100 / 200 ms extra delay	Time rises; slow share falls	7.8 / 10.7 / 12.2 / 14.4 / 16.0 s; slow path 58 % to 16 %	Pass
E4	3 paths; path 2 killed at 1.5 s	Completes, hash verified	Success; 8.5 s vs 6.2 s; 1 retransmission; recovery 0.15 s	Pass
E5	Chunk corruption 0 / 1 / 5 %, 4 paths	All succeed	0 / 2 / 12 retransmitted of 200 chunks; 4.7 / 4.8 / 5.0 s	Pass
E6	Chunk size 64 KiB to 4 MiB	Larger chunk, higher goodput, then flat	50.2 / 90.0 / 107.0 / 107.1 Mbit/s	Pass
E7	Sender killed at 40 %, rerun	Only missing chunks sent	Killed at 3.8 s with 80/200 held; resumed run 5.7 s vs 9.2 s; 80 skipped	Pass

Browser behaviour was tested separately with Playwright: two real instances (alice and bob) transfer a 30 MB file while a path is killed, and the test checks that both sides record the same event stream, that the graphs draw, and that the received file can be downloaded.

Scenario tests (tests/e2e/test_scenarios.py) check that the sender and receiver pages differ (title, banner, panel order, Download only on the receiver), that a short transfer gets a millisecond axis with a real curve and in-range tick labels, that the slow-WAN scenario succeeds, and that the total-failure scenario shows Not delivered on the sender, Nothing was saved on the receiver and leaves no file on disk. The final run had 75 passing tests (50 unit and integration, 25 browser).

4.2 Results and Execution Evidence

The graphs below are produced from the CSV files by experiments/plot_results.py. Figure 4.1 shows near-linear scaling of goodput with the number of paths. Figure 4.2 shows that adding delay to one path raises total time while the transfer still completes and the slow path's share of bytes falls. Figure 4.3 shows the byte share for paths of 80, 20 and 5 Mbit/s.

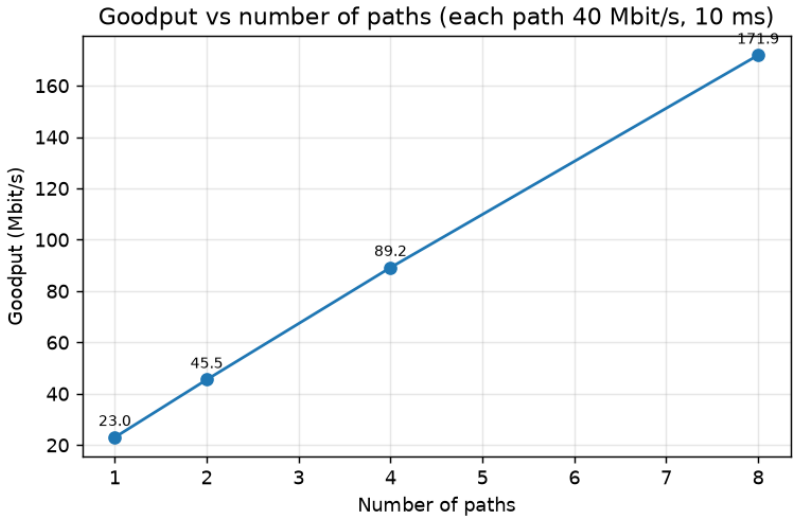


Figure 4.1 Goodput versus number of paths (each 40 Mbit/s, 10 ms)

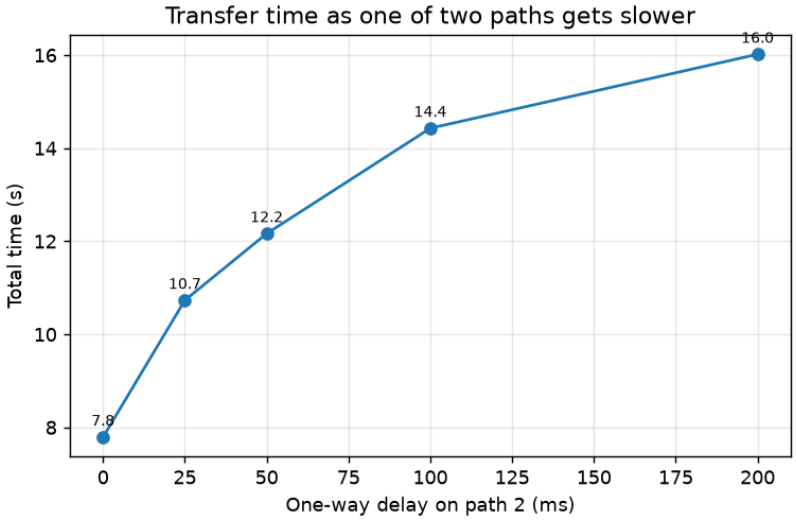


Figure 4.2 Transfer time versus extra delay on one of two paths

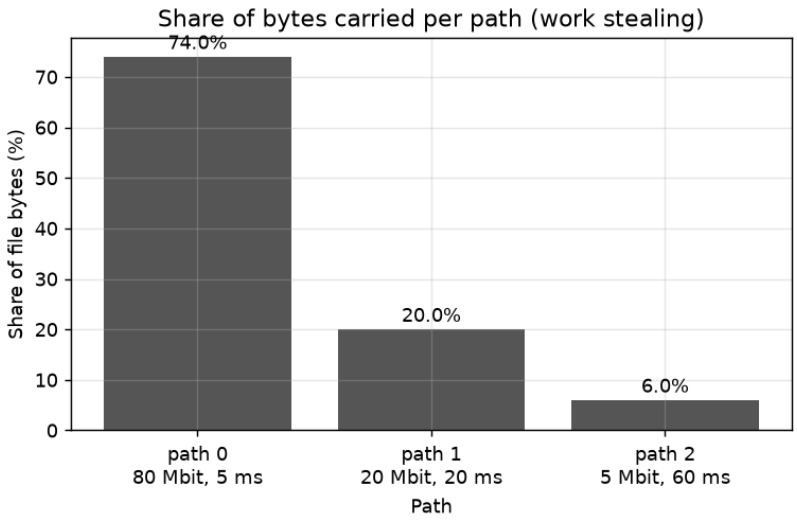


Figure 4.3 Bytes carried per path (80 / 20 / 5 Mbit/s)

Figure 4.4 shows the failure experiment: path 2 dies at 1.5 s, its in-flight chunk is resent on another path 0.15 s later, and the transfer ends with a verified hash 2.3 s later than the undisturbed run, which is the lost capacity rather than recovery delay. Figure 4.5 shows that the number of retransmissions follows the injected corruption rate (2 and 12 of 200 chunks at 1 % and 5 %). Figure 4.6 shows the effect of chunk size.

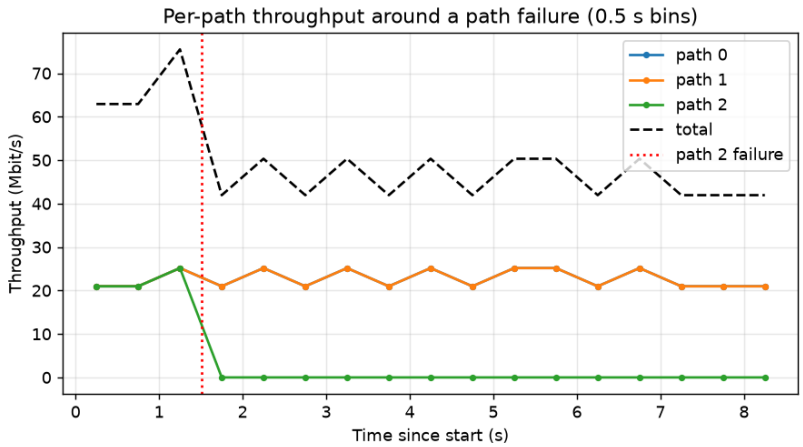


Figure 4.4 Recovery timeline after a path failure

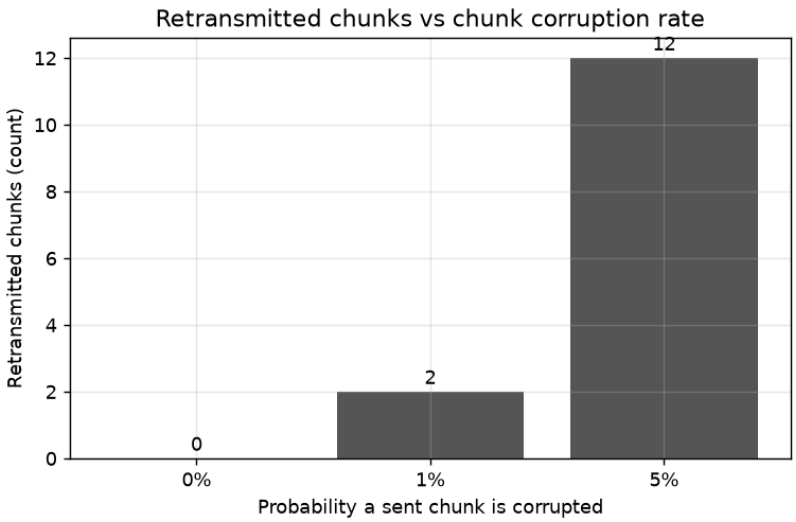


Figure 4.5 Retransmitted chunks versus corruption rate

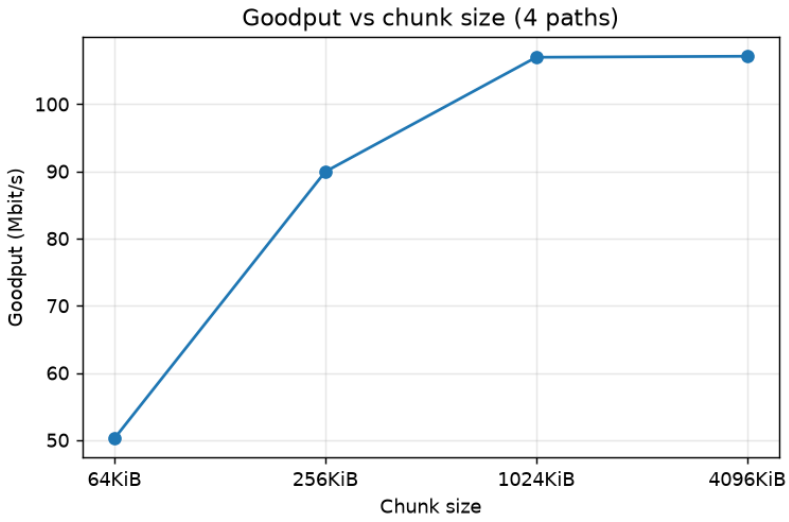


Figure 4.6 Goodput versus chunk size

4.3 Objective Achievement

Table 4.3 links each objective to the tests above. O1 to O5 were demonstrated on emulated paths on one machine; nothing in this report claims a real multi-network result.

Table 4.3 Objective achievement

Objective ID	Evidence or Test ID	Outcome
O1	T3, E1	Achieved (emulated paths)
O2	T1, T4, T8, E5	Achieved
O3	T6, T7, T9, E4	Achieved; silent path costs up to the 5 s chunk timeout
O4	T11, E2, E3	Achieved
O5	T10, E7	Achieved
O6	T13, T14, Playwright tests, tshark loopback capture	Achieved; not tried on a real Wireshark GUI session or two Macs

5 Discussion and Conclusion

5.1 Discussion and Comparative Analysis

Scaling. Goodput roughly doubles whenever the number of paths doubles (E1). Comparing one path with eight, the same file went from 18.2 s to 2.4 s, a 7.5x speed-up, which is the benefit of using several paths over a single connection. A single 40 Mbit/s path reached only 23 Mbit/s because only one chunk is in flight per path, so every chunk pays transmit time plus a round trip; a sliding window per path would fix this and was left out to keep the protocol explainable.

Load sharing. Work stealing needs no bandwidth measurement: the 80 Mbit/s path carried 74 % of the bytes (E2). Total time is still longer than the sum of link rates would suggest, because of the per-chunk round trip and the tail when the last chunk sits on a slow path.

Failure. Losing one of three paths added about 2.3 s, matching the lost capacity; recovery of the in-flight chunk took 0.15 s (E4). Making retransmitted chunks go before fresh ones reduced recovery from 6.9 s to 0.22 s in an earlier run.

Chunk size. 64 KiB chunks lost almost half the goodput to round trips; there was no gain beyond 1 MiB here, while round-trip time per chunk grew from 41 ms to 1040 ms (E6), so a retransmission becomes more costly.

Compared with a single connection, MPFT trades protocol simplicity for throughput and recovery; the comparison is measured only between MPFT runs with different path counts, not against scp or a real MPTCP stack.

5.2 Advantages and Applications

MPFT demonstrates higher throughput with more paths, bit-exact delivery despite corruption and path loss, and resumption without resending verified data. Such behaviour applies to moving large files between hosts with several interfaces (Wi-Fi plus Ethernet, or LAN plus VPN), to unreliable links where a restart from zero is expensive, and as a teaching tool: the dashboard and the Wireshark dissector make each protocol step visible.

5.3 Limitations

- **Evidence:** one machine, emulated paths, one run per setting; no confidence intervals, so small differences are within noise.

- **Emulation level:** the proxy works above TCP and does not drop IP packets, so real TCP loss behaviour is not tested.
- **Protocol:** one chunk per path is in flight at a time, and a silent path costs up to the 5 s chunk timeout.
- **Security:** no encryption or authentication; the web UI binds to 127.0.0.1 and the peer API is meant for a trusted LAN or Tailscale.
- **Untested:** a two-machine run over Tailscale and LAN was implemented (docs/REMOTE.md) but is not part of the measurements; the browser interface was tested with Chromium only.
- **Test flakiness:** one emulator corruption test occasionally fails when run repeatedly (about 1 run in 8) because corruption is random.

5.4 Challenges and Troubleshooting

These problems occurred during development and are recorded in PROGRESS.md.

Table 5.1 Problems found and how they were resolved

Issue	Cause or Diagnosis	Solution and Verification
Recovery after a path died took 6.9 s	Retried chunks went to the back of the FIFO queue behind fresh ones	Priority queue: retries first; recovery 0.22 s, later measured 0.15 s (E4)
Corruption percentage did not match expectations	Emulator corrupts per 16 KiB block, not per chunk	Experiments inject per-chunk corruption in the sender; 1 % and 5 % gave 2 and 12 retransmissions of 200
A damaged chunk in file.part would have been trusted on resume	Resume only counted stored chunks	Receiver re-hashes each stored chunk against the manifest; test U14 passes (15 of 16 resumed)
Graphs missing or clipped in the web dashboard	Single-point lines invisible, clipped x range, overlapping labels, no redraw on resize	Fixed in the graph code; checked by Playwright e2e tests
Browser test results polluted by earlier runs	Orphaned servers left by the test harness	Process groups and a port check in the harness; event streams on both sides identical

5.5 Future Scope

- A sliding window per path to remove the one-round-trip-per-chunk limit.
- Adaptive chunk size per path.
- A real two-machine run with Wi-Fi and Ethernet, with Wireshark captures on both sides.
- Encryption and sender authentication, for example TLS per path.
- A comparison with scp and with kernel MPTCP.
- Repeated runs with confidence intervals.

5.6 Conclusion

MPFT is a working multipath file-transfer protocol built from standard-library Python. Against the objectives: a file is split across parallel TCP paths and reassembled correctly (O1); corruption is caught at header, chunk and file level and repaired (O2); path failure and silent paths are tolerated within a bounded retry policy (O3); faster paths carry more data without measuring bandwidth (O4); and interrupted transfers resume (O5). The protocol is visible through the dashboard and a Wireshark dissector (O6). The measured scaling from 23.0 to 171.9 Mbit/s with eight paths, hash-verified completion after a mid-transfer path failure, and 80 chunks skipped on resume support these claims, on emulated paths on one machine. The main limitations are the stop-and-wait design and the absence of a real multi-network test.

References

Numbered in order of first citation.

[1] J. Eddy, “Transmission Control Protocol (TCP),” RFC 9293, IETF, Aug. 2022. https://www.rfc-editor.org/rfc/rfc9293

[2] M. Allman, V. Paxson and E. Blanton, “TCP Congestion Control,” RFC 5681, IETF, Sep. 2009. https://www.rfc-editor.org/rfc/rfc5681

[3] A. Ford, C. Raiciu, M. Handley, O. Bonaventure and C. Paasch, “TCP Extensions for Multipath Operation with Multiple Addresses,” RFC 8684, IETF, Mar. 2020. https://www.rfc-editor.org/rfc/rfc8684

[4] C. Raiciu, C. Paasch, S. Barre, A. Ford, M. Honda, F. Duchene, O. Bonaventure and M. Handley, “How hard can it be? Designing and implementing a deployable Multipath TCP,” in Proc. 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2012, pp. 399–412.

[5] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, Aug. 2015.

[6] Python Software Foundation, “socket — Low-level networking interface” and “hashlib — Secure hashes and message digests,” Python 3 documentation. https://docs.python.org/3/library/

[7] Wireshark Foundation, “Wireshark Developer’s Guide: Lua Support in Wireshark.” https://www.wireshark.org/docs/wsdg_html_chunked/wsluarm.html

AI Usage Declaration

AI tools were used. The group used Claude Code, an AI coding assistant from Anthropic, to help write and test the MPFT source code, the experiment scripts, the web interface and the documentation, and to draft this report and the presentation from the project's own files. All measurements in this report were produced by running the code (the experiment CSV files in results/ and the pytest run described in Section 4.1); they were not generated by the AI tool. Statements about untested conditions, such as the two-machine run, are marked as untested. The group is responsible for reading the code and this report and for being able to explain them.

Declaration: We take responsibility for the accuracy of this report and understand the submitted implementation and results. Sources and assistance used have been acknowledged.

Student Name	PRN	Signature
Saksham Singh Sisodiya	1262242908	
Parth Dande	1262242629	

Appendices

Appendix A Code and Configuration

Full source is in the repository (mpft/, web/, experiments/, tests/, wireshark/). The table lists the main files; mpft/protocol.py, which defines the wire format, follows in full, then the example emulator configuration. No secrets or credentials are included.

Table A.1 Main source files

File	Lines	Purpose
mpft/protocol.py	95	Header, framing, manifest
mpft/sender.py	264	Path threads, queue, retries, metrics
mpft/receiver.py	183	Verification, offset writes, resume, final rename
mpft/emulator.py	162	Delay, jitter, rate, kill, corrupt proxy
mpft/emu_control.py	42	HTTP control for the emulator
mpft/status.py	50	Status JSON writer
mpft/events.py	50	Event bus for the dashboard
mpft/cli.py	107	send, receive, emulate, dashboard, web commands
wireshark/mpft.lua	n/a	Wireshark dissector
experiments/	n/a	Experiment runner and plots

```
"""MPFT wire format: 20-byte header + payload. See docs/PROTOCOL.md."""
import hashlib
import json
import os
import socket
import struct
import zlib

MAGIC = b"MP"
VERSION = 1
HELLO, MANIFEST, MANIFEST_ACK, CHUNK, ACK, NACK, DONE, ERROR = range(1, 9)
TYPE_NAMES = {1: "HELLO", 2: "MANIFEST", 3: "MANIFEST_ACK", 4: "CHUNK",
              5: "ACK", 6: "NACK", 7: "DONE", 8: "ERROR"}

# magic, version, type, session_id, chunk_index, payload_len; crc32 follows
_BODY = struct.Struct(">2sBBIII")
HEADER_LEN = _BODY.size + 4  # 20
MAX_PAYLOAD = 64 * 1024 * 1024  # sanity limit so a corrupt length can't make us allocate GBs


class ProtocolError(Exception):
    """Malformed frame (bad magic, version, CRC or length). The stream is unusable afterwards."""


def pack_header(mtype, session, index, payload_len):
    body = _BODY.pack(MAGIC, VERSION, mtype, session, index, payload_len)
    return body + struct.pack(">I", zlib.crc32(body))


def unpack_header(raw):
    if len(raw) != HEADER_LEN:
        raise ProtocolError("short header")
    body, (crc,) = raw[:-4], struct.unpack(">I", raw[-4:])
    if zlib.crc32(body) != crc:
        raise ProtocolError("header CRC mismatch")
    magic, version, mtype, session, index, plen = _BODY.unpack(body)
    if magic != MAGIC or version != VERSION or mtype not in TYPE_NAMES:
        raise ProtocolError("bad magic/version/type")
    if plen > MAX_PAYLOAD:
        raise ProtocolError("payload too large")
```

```
    return mtype, session, index, plen


def send_msg(sock, mtype, session, index=0, payload=b""):
    sock.sendall(pack_header(mtype, session, index, len(payload)) + payload)


def recv_exact(sock, n):
    buf = bytearray()
    while len(buf) < n:
        part = sock.recv(n - len(buf))
        if not part:
            raise ConnectionError("peer closed connection")
        buf += part
    return bytes(buf)


def recv_msg(sock):
    """Return (type, session, index, payload)."""
    mtype, session, index, plen = unpack_header(recv_exact(sock, HEADER_LEN))
    return mtype, session, index, recv_exact(sock, plen) if plen else b""


def send_json(sock, mtype, session, obj, index=0):
    send_msg(sock, mtype, session, index, json.dumps(obj).encode())


def parse_addr(text):
    host, _, port = text.rpartition(":")
    return host or "0.0.0.0", int(port)


def chunk_len(size, chunk_size, index):
    return min(chunk_size, size - index * chunk_size)


def sha256_hex(data):
    return hashlib.sha256(data).hexdigest()


def build_manifest(file_path, chunk_size):
    """One streaming pass: per-chunk hashes and whole-file hash, never the whole file in RAM."""
    size = os.path.getsize(file_path)
    whole, chunks = hashlib.sha256(), []
    with open(file_path, "rb") as f:
        while data := f.read(chunk_size):
            whole.update(data)
            chunks.append(sha256_hex(data))
    return {"name": os.path.basename(file_path), "size": size, "chunk_size": chunk_size,
            "total_chunks": len(chunks), "file_sha256": whole.hexdigest(), "chunk_sha256":
chunks}


def tune(sock):
    sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
    return sock
```

```
// topology.example.json
[
  {"listen_port": 9201, "to": "127.0.0.1:9000", "delay_ms": 2,  "jitter_ms": 0,  "rate_mbit": 0,
"drop_after_s": 0, "corrupt_rate": 0},
  {"listen_port": 9202, "to": "127.0.0.1:9000", "delay_ms": 20, "jitter_ms": 5,  "rate_mbit": 20,
"drop_after_s": 0, "corrupt_rate": 0},
  {"listen_port": 9203, "to": "127.0.0.1:9000", "delay_ms": 40, "jitter_ms": 10, "rate_mbit": 8,
"drop_after_s": 6, "corrupt_rate": 0}
]
```

Appendix B Additional Evidence

Table B.1 gives the byte split per path and chunk RTT for selected runs, taken from results/ *.csv. The tshark check decoded a real loopback capture with mpft.lua without a manual Decode As step; the display filters are mpft.type == 4 for chunks.

Table B.1 Selected raw results

Run	Time (s)	Goodput (Mbit/s)	Retransmitted chunks	Bytes per path
E1, 1 path	18.2	23.0	0	52428800
E1, 8 paths	2.4	171.9	0	6553600 each
E2, 80/20/5	7.6	55.0	0	38797312 / 10485760 / 3145728
E4, no failure	6.2	67.2	0	17301504 / 17563648 / 17563648
E4, path 2 dies at 1.5 s	8.5	49.3	1	24117248 / 24117248 / 4194304
E5, 5 % corruption	4.9	84.8	12	about 13.1 MB each
E7, uninterrupted	9.2	45.6	0	26214400 each

Appendix C Individual Contributions

Work is split by module so that each member can explain their part in the viva.

Table C.1 Individual contributions

Student Name and PRN	Work Completed	Evidence or Deliverable
Saksham Singh Sisodiya, PRN 1262242908	Protocol, sender and receiver; resume; retry and backoff; web interface and dashboard; documentation	mpft/protocol.py, sender.py, receiver.py, cli.py, web/, docs/PROTOCOL.md, docs/ARCHITECTURE.md
Parth Dande, PRN 1262242629	Network emulator and experiments; Wireshark dissector and capture steps; test report; presentation	mpft/emulator.py, experiments/, results/, wireshark/, docs/TEST_REPORT.md